Signed Eight Bit Calculator

Neel Raje, Samuel Robin
Electrical and Computer Engineering Department
School of Engineering and Computer Science
Oakland University, Rochester, MI

***Abstract*—This report describes the design, simulation, and hardware implementation of a signed eight bit calculator built entirely in structural and behavioral VHDL and deployed on a Nexys A7 100T board. The system accepts two signed eight bit operands from the board switches, performs addition, subtraction, multiplication, or division using dedicated datapath hardware, and displays the signed result on an eight digit multiplexed seven segment display. The design combines a finite state machine controller with a datapath built from gate level components, including a ripple carry adder, an array multiplier, and a restoring division circuit. All operations were verified first in behavioral simulation against a set of nine test cases, including operand values at the extreme edge of the signed range and a divide by zero condition, then confirmed on physical hardware.**

I. INTRODUCTION

This project was completed for a digital logic design course covering combinational and sequential circuit design in VHDL, and it draws on most of what the course covers: gate level structural modeling, generate statements, finite state machines, registered datapaths, and board level constraints. The scope was to build something that resembles a calculator a person could use, not just a testbench exercise, which meant handling negative numbers correctly, avoiding overflow on the extreme input values, and detecting divide by zero rather than letting the hardware do something undefined.

Two elements of this design went beyond what the course covered and were learned independently while building it: the restoring division algorithm used for the divide operation, and the double dabble method for converting a binary magnitude into binary coded decimal digits for display. Both are described in Section II.

The rest of this report walks through the datapath component by component, then the controlling state machine, then the test methodology and results, and closes with a discussion of what worked and what did not.

II. METHODOLOGY

A. System Overview

The top level entity, top.vhd, wires together a controller and a datapath. Two eight bit operands come in on SW15 through SW8 for operand A and SW7 through SW0 for operand B, both read as two's complement values. Four buttons select the operation and a fifth, BTNC, both captures the operands and starts the calculation. Everything downstream of that button press, from capturing A and B to lighting the correct segments, is driven off the same finite state machine, described in Fig. 1.

B. Input Capture and Sign Magnitude Conversion

Once BTNC is pressed, the FSM asserts LAB, an enable signal that loads A and B into two eight bit registers, my_rege, built from a bank of enabled D flip flops with an asynchronous active low reset. From there the design splits every value into a sign bit and an unsigned magnitude using my_abs (Fig. 6, left half), because both the array multiplier and the division circuit only work correctly on positive magnitudes. My_abs passes the value through unchanged when the top bit is zero and computes the two's complement (invert then add one) when the top bit is one. This same block gets instantiated three separate times in top.vhd: once for the raw switch value of A, once for B, and once for the nine bit result coming out of the adder subtractor.

C. Addition and Subtraction

Addition and subtraction share one piece of hardware, my_addsub, built as a chain of nine full adders (Fig. 3), each one identical to the gate level full adder in Fig. 2. A single control bit, addsub, decides which operation runs: it is XORed into every bit of B before the adder sees it, and it also becomes the initial carry in. When addsub is zero, B passes through unchanged and the carry chain starts at zero, giving ordinary addition. When addsub is one, B is inverted and the carry starts at one, which is exactly the two's complement recipe for subtraction, so no separate subtractor circuit was needed.

This unit runs at nine bits rather than eight, with both operands sign extended before entering the adder. That extra

bit exists specifically because of the edge cases in the test set. Negative 128 plus negative 128 equals negative 256, and 127 minus negative 128 equals 255, and neither of those fits in an eight bit signed result. This was caught during simulation, before it reached hardware: an eight bit only version would have silently wrapped around on exactly the inputs the test table calls for.

D. Array Mutliplier

Multiplication uses my_arrmult, an eight by eight unsigned array multiplier built from sixty four identical processing unit cells, PU.vhd, arranged in an eight row by eight column grid (Fig. 4). Each PU cell is one AND gate feeding a full adder, shown on the right side of Fig. 2. The AND gate produces one partial product bit, and the full adder combines it with a carry coming from the cell to its left and a sum bit coming diagonally from the row above. That diagonal connection is what performs the shift and add of long multiplication in hardware: every row multiplies by one bit of B and adds the shifted result into a running total, without ever using a register or a clock cycle to do it. The whole multiplier is a purely combinational block, which is part of why the FSM does not need to wait on it, it just reads the answer once the inputs have settled.

Since the multiplier only understands magnitudes, the sign of the sixteen bit product is recovered separately as the exclusive or of the two input signs, the standard rule for signed multiplication once sign and magnitude have been factored apart.

E. Restoring Division

Division is the only operation in the design that needs multiple clock cycles, and its datapath is the most involved part of the whole design. It uses a restoring division algorithm built from two shift capable registers (my_pashiftreg_sclr, right side of Fig. 5) and a reused instance of my_addsub hardwired to always subtract.

Here is the loop, one iteration per clock cycle for eight cycles total. The top bit of the dividend register, referred to in the code as a7, is shifted into the bottom of the remainder register, forming a nine bit trial value. That trial value has the divisor subtracted from it. If the subtraction does not underflow, meaning the remainder was at least as large as the divisor, the subtraction result becomes the new remainder and a one shifts into the dividend register. If it does underflow, the remainder is left alone for that cycle, the restoring step, and a zero shifts in instead. After eight cycles the dividend register, which started out holding the magnitude of A, has been completely shifted out one bit at a time and has an eight bit quotient shifted in behind it, so one register does double duty and no separate quotient register was needed. A small counter, my_genpulse_sclr (Fig. 6, right half), counts the eight iterations and tells the state machine when to stop.

The divider works through the dividend one bit at a time rather than all at once, which makes it a bit serial circuit in the same sense a UART or a shift register based multiplier is serial. It trades hardware for time, spending eight clock cycles instead of eight parallel copies of the subtract logic. The seven segment display, described later in this section, has a second and unrelated serializer of its own, converting eight parallel digit values into one serial stream over time. Both exist for the same underlying reason, less hardware in exchange for more clock cycles, even though one comes from the arithmetic side of the design and the other from the display side.

Remainder sign follows the sign of the dividend rather than being computed independently, which matches the test case of negative 100 divided by 7 giving a quotient of negative 14 and a remainder of negative 2 rather than a positive one. Quotient sign uses the same exclusive or rule as multiplication.

F. Finite State Machine Control

The controller, my_fsm, has four states and ties every datapath block above into a single sequence. State S1 is idle. It sits here clearing the division registers and waiting for BTNC. The instant BTNC goes high, the same clock edge decides where to go next: to S4 if division was selected and B is zero, to S2 if division was selected and B is not zero, or straight to S3 for addition, subtraction, or multiplication, since those are already combinational and have nothing left to wait on. State S2 is the division loop and stays there until the iteration counter signals it is done, at which point it moves to S3. States S3 and S4 both assert a done pulse, the only difference being that S4 also asserts a divide by zero flag, and both hold their state as long as BTNC stays pressed, returning to S1 the moment the button is released.

Add, subtract, and multiply do not need their own states, since each is already combinational and finishes the instant, its inputs are captured. Folding all three into the same transition out of S1 keeps the state count at four rather than six or seven, which also keeps the state diagram simple enough to check by hand against the test cases.

G. BCD Conversion and Display Serialization

Results come out of the datapath as raw binary magnitudes, which are not directly displayable as decimal digits, so each result path, addition and subtraction, multiplication, quotient, and remainder, runs through its own instance of

my_bin2bcd, using the double dabble algorithm: shift the binary value left one bit at a time into a set of four bit decimal digit fields, adding three to any digit field that exceeds four before each shift. This runs combinationally off the held result and produces the digits shown on the display.

On a single done pulse, all of these results, along with the operation that was selected and the divide by zero flag, get latched into a bank of hold registers. Without this step, the display would be driven directly by signals that are still settling combinationally in the cycles right after the button is released, so the hold registers exist specifically to keep the displayed digits stable between calculations.

The eight digit seven segment display only has one shared set of decoder inputs, so my_7seg_scan is what gets all eight digits onto the screen. It cycles through the eight digit positions using the upper bits of a free running counter, driving one anode low at a time while feeding that digit's four bit code to the decoder. In effect it is a parallel to serial converter, taking eight parallel digit codes and serializing them onto one shared bus over time, fast enough, a little over six kilohertz per digit, that the eye reads it as eight digits lit at once rather than one digit blinking rapidly. My_char7seg is the final decoding step, turning each four bit code into the seven segments, with three special codes beyond the ten digits: a blank code that turns a digit position off, a minus sign, and the letter E used for the divide by zero display.

III. EXPERIMENTAL SETUP

All thirteen design files were added to a Vivado project targeting the Nexys A7 100T, with top_tb.vhd added separately as a simulation source and calculator.xdc providing the pin constraints. Before moving to hardware, a behavioral simulation was run through a sequence of nine stimulus cases, each one selecting an operation, pulsing BTNC, and waiting long enough for a done pulse before the next case began, with internal signals such as the FSM state and the held BCD outputs checked at each done pulse.

After synthesis and implementation completed without errors, a bitstream was generated and loaded onto the board through Vivado's hardware manager. Operand values were set with the sixteen switches, an operation was chosen with one of the four directional buttons, and BTNC triggered the calculation. The seven segment output was read by hand for each case in Table I, and it matched the expected value every time.

IV. RESULTS

Table I lists the nine cases used to verify the design, each one confirmed directly on the physical board.

Test	Expected	Result
45 + 30	+75	+00075
-128 + -128	-256	-00256
50 - 90	-40	-00040
127 – (-128)	+255	+00255
12 × −10	-120	-00120
−128 × −128	+16384	+16384
100 ÷ 7	Q14 R2	Q+014 R+002
−100 ÷ 7	Q-14 R-2	Q-014 R-002
50 ÷ 0	Error	Error

Table 1

Q denotes quotient and R denotes remainder for the division cases.

Three of the nine cases use negative 128 as an operand: once in addition, once in subtraction, and once in multiplication. Negative 128 has no positive counterpart in eight bit two's complement, so these cases specifically check that the sign magnitude conversion does not break on that one value, and all three produced correct results. The divide by zero case correctly forced the display into its error state rather than producing a meaningless quotient.

V. CONCLUSIONS

This system shows how much of a full design, capture, arithmetic, control, and display, can be built from a fairly small library of reusable structural components. The full adder shows up inside the ripple carry adder, inside every cell of the array multiplier, and inside the division subtractor, and the same parallel load shift register handles both operands of the divider with only its control inputs changed.

There are a few places the design could be tightened. The operation select latch has an implicit priority order between the four buttons, since it checks them in a fixed sequence, and it does not do anything special if two are pressed on the same clock edge, which was not tested for and would be worth handling directly in a revision. A non restoring division algorithm would also cut the division latency

roughly in half, at the cost of a state machine somewhat more complicated than the four state one used here.

Overall the design meets its goal of a working signed calculator, verified in simulation and confirmed on hardware, and it required working through several sequential design techniques, restoring division and multiplexed display driving in particular, that go beyond what this course explicitly covers.

VI. REFERENCES

[1] Digilent Inc., Nexys A7 Reference Manual

[2] Xilinx Inc., Vivado Design Suite User Guide, Synthesis.

FIGURES

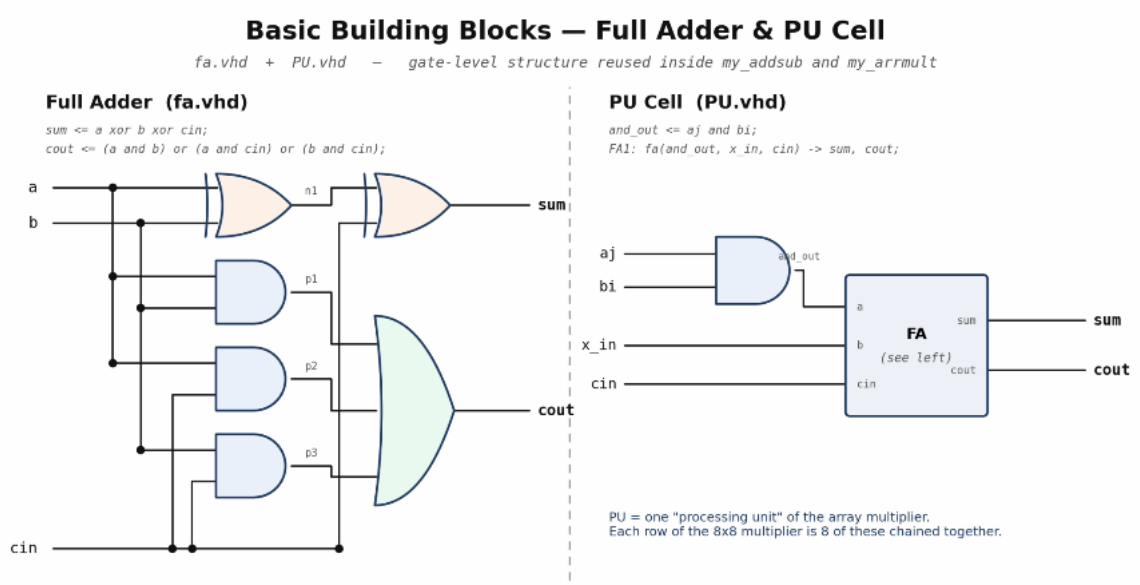

Fig. 1*. Top level block diagram of top.vhd, showing the control unit, the signed ALU and datapath, and the BCD converter and seven segment driver, with inputs on the left and outputs on the right.*

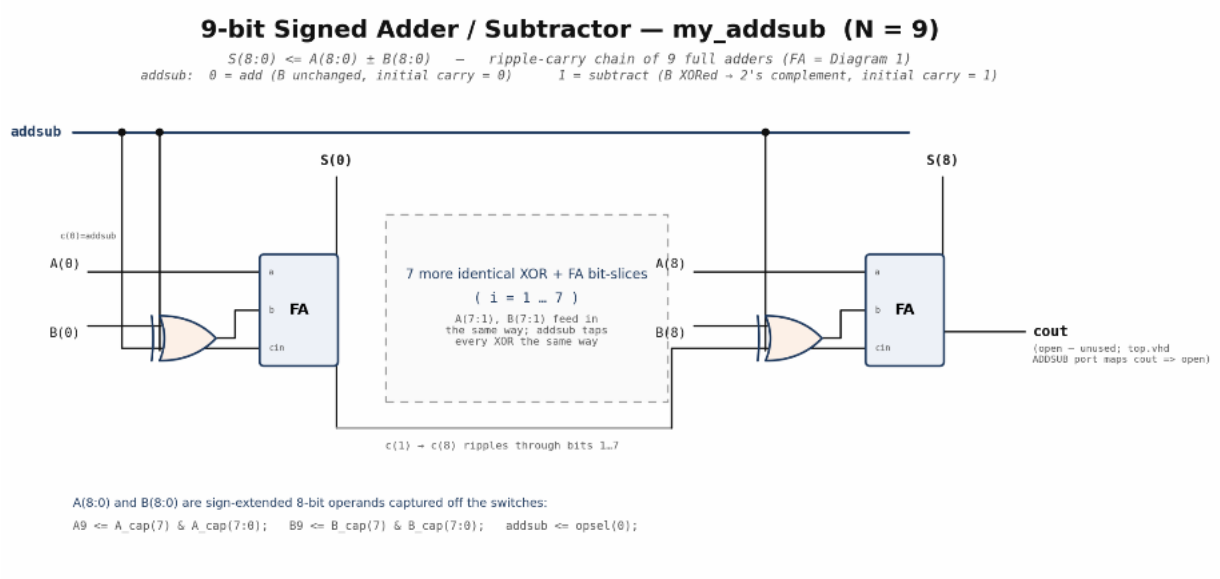

Fig. 2. Gate level structure of the full adder (fa.vhd) alongside the processing unit cell (PU.vhd) that reuses it, the basic building block instantiated throughout both my_addsub and my_arrmult.

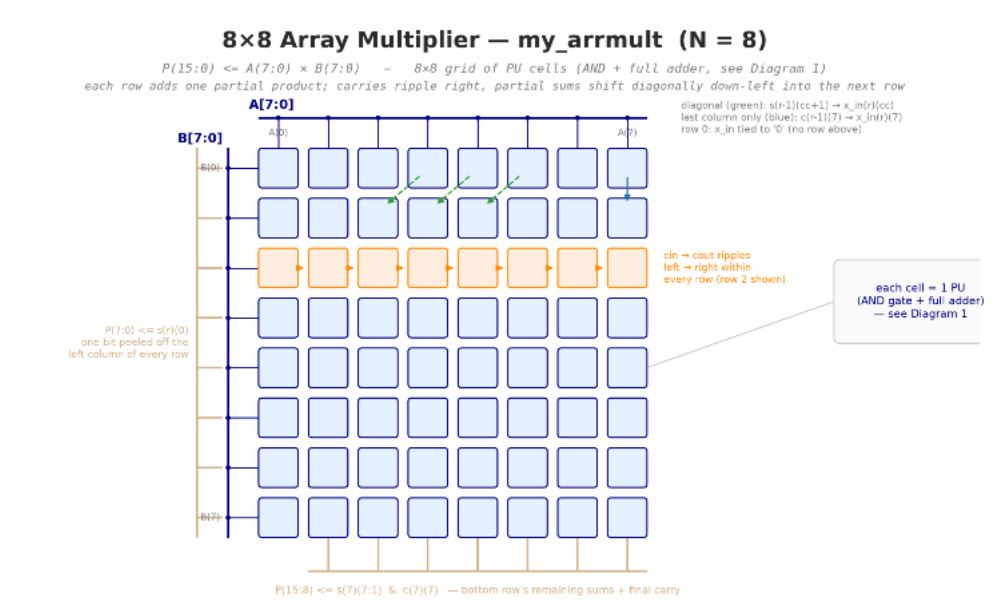

Fig. 3. The nine bit adder subtractor (my_addsub), a ripple carry chain of nine full adders with the addsub control bit XORed into B and fed in as the initial carry.

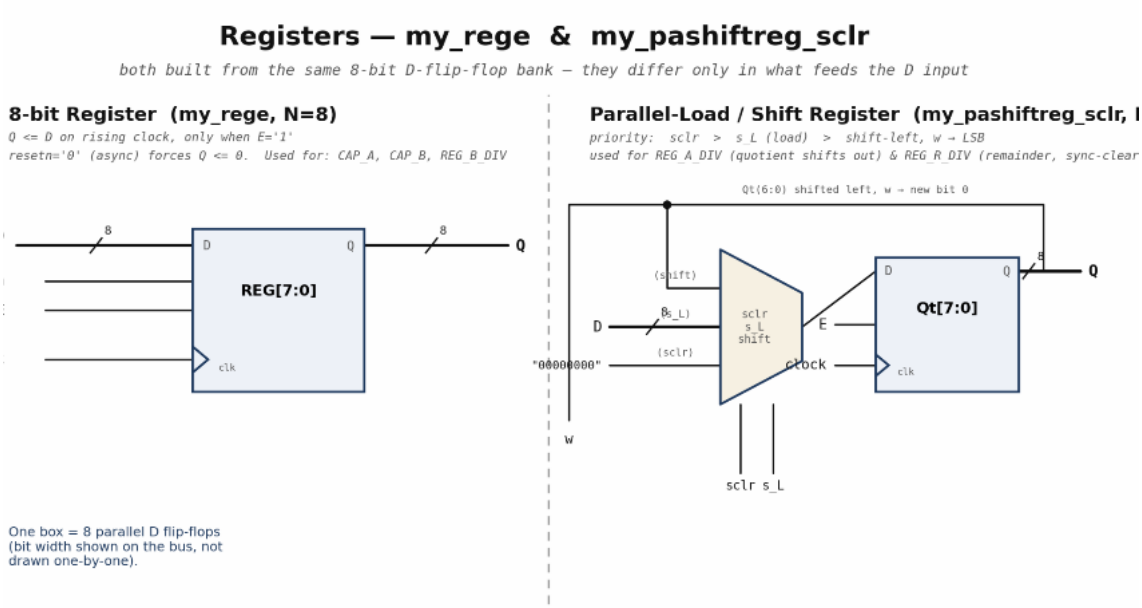

Fig. 4. The eight by eight array multiplier (my_arrmult), showing the grid of PU cells with carries rippling right within each row and partial sums shifting diagonally down and left into the next row.

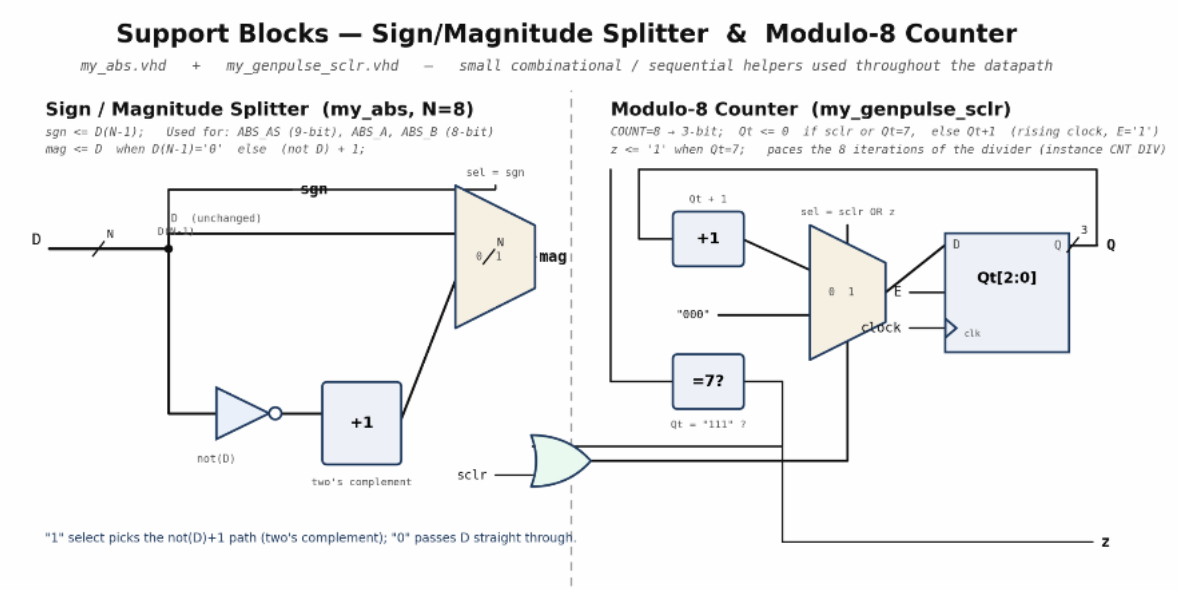

Fig. 5. The two register types used throughout the design, my_rege and my_pashiftreg_sclr, both built from the same D flip flop bank and differing only in what feeds the D input.

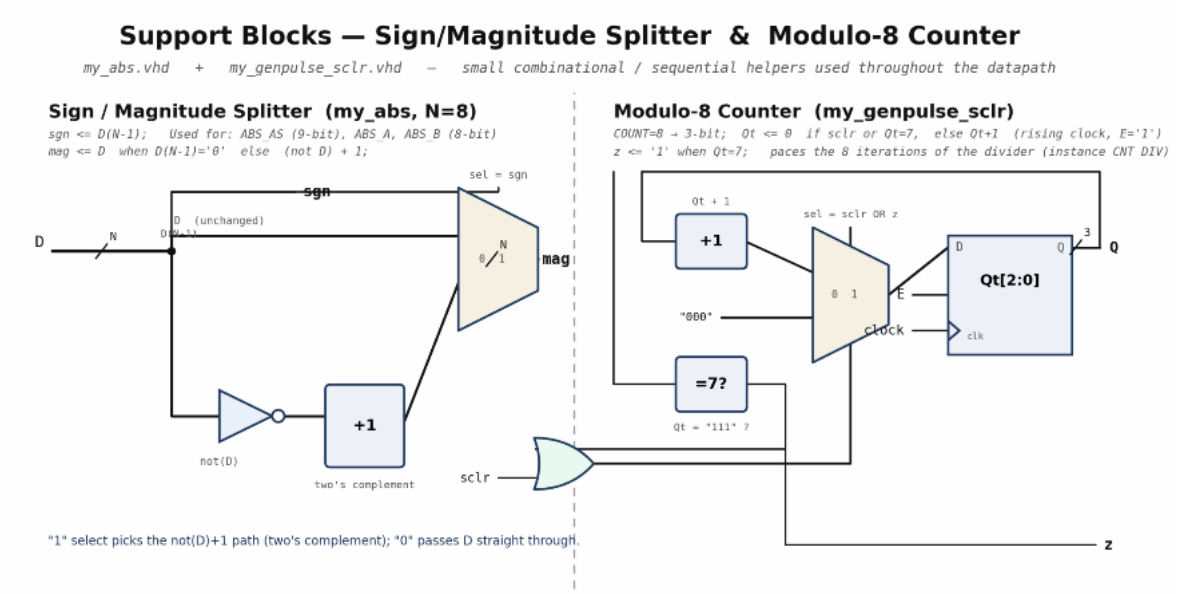

Fig. 6. The sign magnitude splitter (my_abs) and the modulo eight counter (my_genpulse_sclr) that paces the eight iterations of the divider.